LAPORAN PRAKTIKUM

INTERNET OF THINGS II

TUGAS 04-05



Nama :

Nama : Fadillah Akbar

NPM : 25782039

Kelas : TRI 3B

Dosen Pengampu : **Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.**

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, A.Md

JURUSAN TEKNOLOGI INFORMASI

TEKNOLOGI REKAYASA INTERNET

POLITEKNIK NEGERI LAMPUNG

TAHUN 2026

4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik

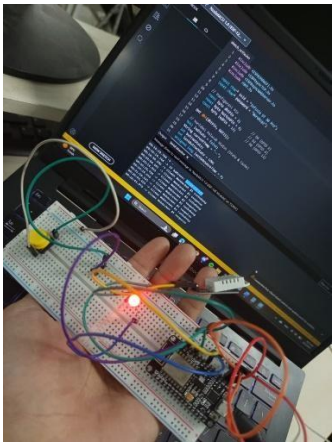
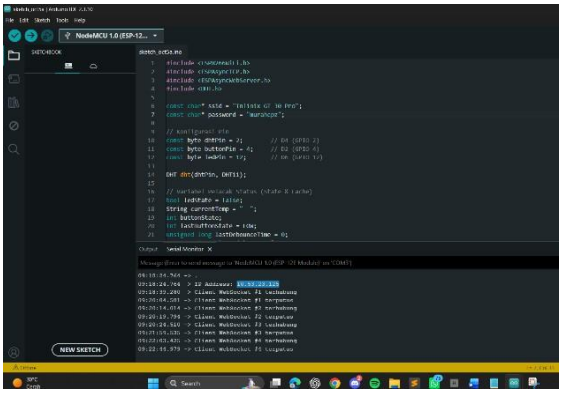
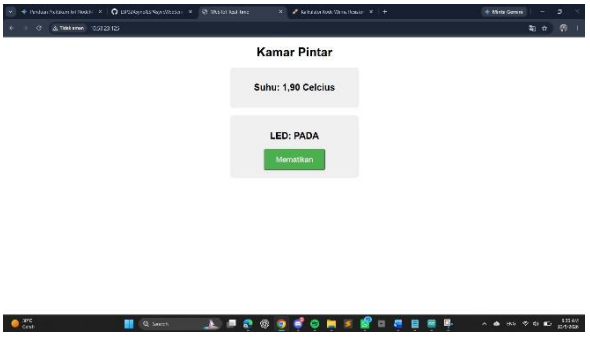


Foto Alat



Ss an Se rial Monitor



Hasil WEB

4.6 Latihan

1. **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawab:

- Efisiensi Bandwidth/Data: Pada metode Meta Refresh HTTP, browser harus berulang kali mengirimkan HTTP Request Header yang berukuran cukup besar dan mengunduh ulang seluruh struktur dokumen HTML, CSS, serta Script dari awal. Sementara pada WebSocket, setelah proses handshake awal selesai, saluran komunikasi dua arah (full-duplex) tetap terbuka secara persisten. Data dikirim hanya dalam bentuk payload mentah berukuran sangat kecil (beberapa byte teks JSON) tanpa overhead header HTTP yang besar.
- Tampilan Tidak Berkedip: Meta Refresh memaksa browser melakukan reload/muat ulang seluruh halaman web dari nol, sehingga DOM dirender ulang dan memicu efek flicker (berkedip). Sebaliknya, WebSocket bekerja secara asynchronous di latar belakang. Data JSON yang diterima ditangkap oleh JavaScript untuk memperbarui teks pada elemen DOM spesifik secara langsung (innerHTML) tanpa perlu menyegarkan atau memuat ulang halaman.

2. **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks delay(3000) di dalam siklus fungsi void loop() pada draf arsitektur kompleks ini?

Jawab:

Penggunaan delay(3000) di dalam loop() akan menghentikan (*blocking*) eksekusi prosesor NodeMCU selama 3 detik penuh. Bahaya sistematis yang terjadi meliputi:

- **Kehilangan Real-Time Respon:** Tekanan pada push button mekanik yang terjadi saat delay() berlangsung tidak akan terbaca oleh sistem.
- **Keterlambatan/Latensi Komunikasi WebSocket:** Sinyal toggle dari browser tidak dapat diolah seketika oleh NodeMCU, menyebabkan status tombol web dan indikator tidak sinkron (*lag*).

- **Potensi Disconnect & Watchdog Reset (WDT):** Komunikasi background TCP/IP dan antrean paket WebSocket terhambat. Jika *stack* jaringan tidak diproses secara teratur, modul ESP8266 dapat mengalami *timeout* koneksi atau ter-reset otomatis oleh hardware Watchdog Timer (Soft WDT reset).

3. **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawab:

Target ID elemen HTML yang diubah oleh fungsi JavaScript pada blok tersebut adalah:

- **tempValue** → Menyuntikkan teks angka suhu terbaru dari paket data JSON (dataObj.suhu).
- **ledStatus** → Mengubah teks indikator status LED menjadi **"ON"** atau **"OFF"**.
- **toggleBtn** → Mengubah teks label tombol menjadi **"Turn OFF"** atau **"Turn ON"**, serta mengganti warna/gaya tombol melalui btn.className (btn-on atau btn-off).

4. **Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
    a. **Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--";). Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer *non-blocking* millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).

Jawab:

- Tambah Variabel Global Penampung Kelembapan:

```
String currentHum = "--"; // Penampung cache kelembapan
```

- Modifikasi Fungsi notifyClients() (JSON Payload):

```
void notifyClients() {
  String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
  jsonString += "\"suhu\":\"" + currentTemp + "\", ";
  jsonString += "\"hum\":\"" + currentHum + "\"}"; // Menyisipkan data kelembapan
  ws.textAll(jsonString);
}
```

- Modifikasi Pembacaan DHT pada loop() (Timer Non-Blocking millis()):

```
// Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
if ((millis() - lastTime) > 3000) {
  float t = dht.readTemperature();
  float h = dht.readHumidity(); // Baca kelembapan

  if(!isnan(t) && !isnan(h)) {
    currentTemp = String(t);
    currentHum  = String(h); // Simpan ke variabel cache
    notifyClients();         // Push pembaruan suhu & kelembapan otomatis
  }
  lastTime = millis();
}
```

b. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu ( ) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawab:

- Penambahan Elemen Kartu HTML:

```
<div class="card">
  <h2>Kelembapan: <span id="humValue">--</span> %</h2>
</div>
```

- Modifikasi Fungsi JavaScript onMessage(event):

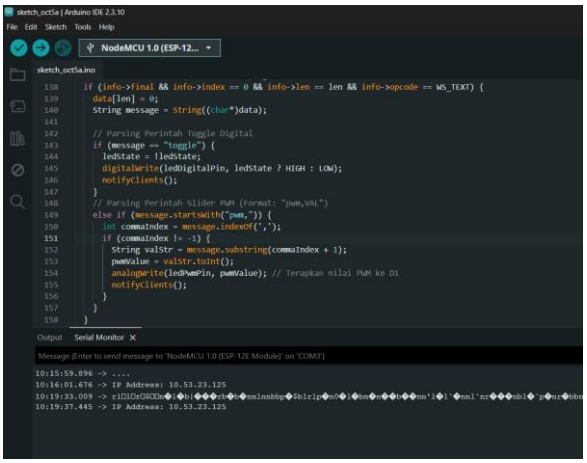
```
function onMessage(event) {
  var dataObj = JSON.parse(event.data);

  // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
  if(dataObj.suhu !== undefined) {
     document.getElementById('tempValue').innerHTML = dataObj.suhu;
  }

  // Menyuntikkan teks angka Kelembapan ke dalam id "humValue" HTML
  if(dataObj.hum !== undefined) {
     document.getElementById('humValue').innerHTML = dataObj.hum;
  }

  // Merombak UI Tombol & Teks sesuai status hardware
  if(dataObj.led !== undefined) {
     var btn = document.getElementById('toggleBtn');
     var status = document.getElementById('ledStatus');
     if(dataObj.led == "1"){
       status.innerHTML = "ON";
       btn.innerHTML = "Turn OFF";
       btn.className = "btn-on";
     } else {
       status.innerHTML = "OFF";
       btn.innerHTML = "Turn ON";
       btn.className = "btn-off";
     }
  }
}
```

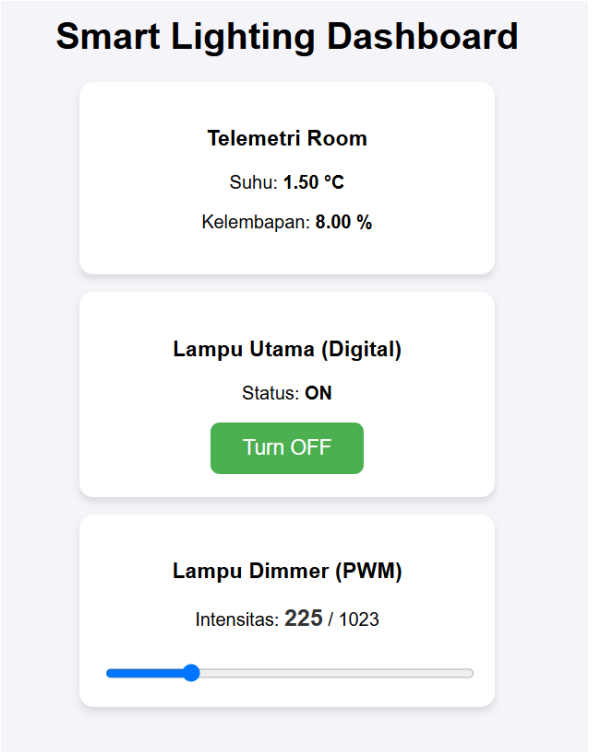
4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*



SS an Serial Monitor

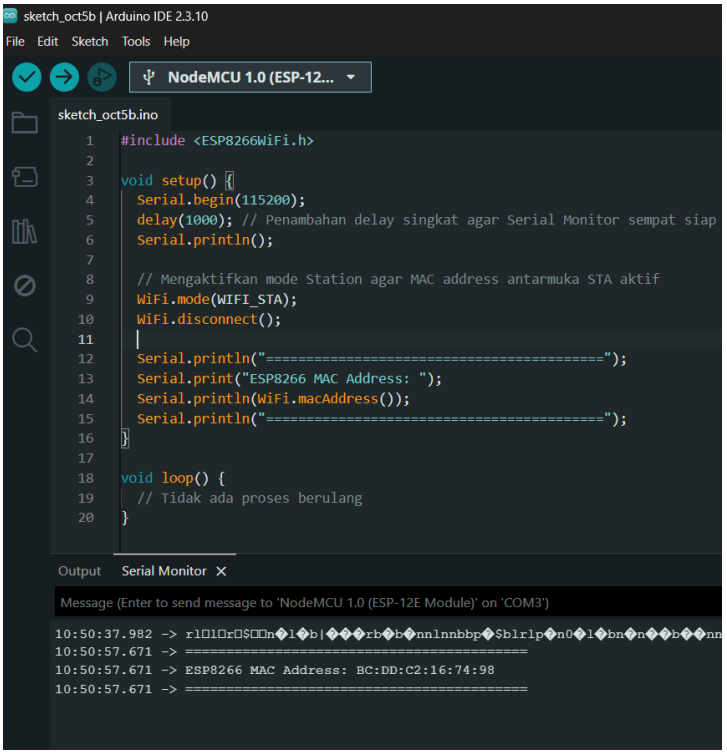


Foto Alat



SS an Tampilan Web

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

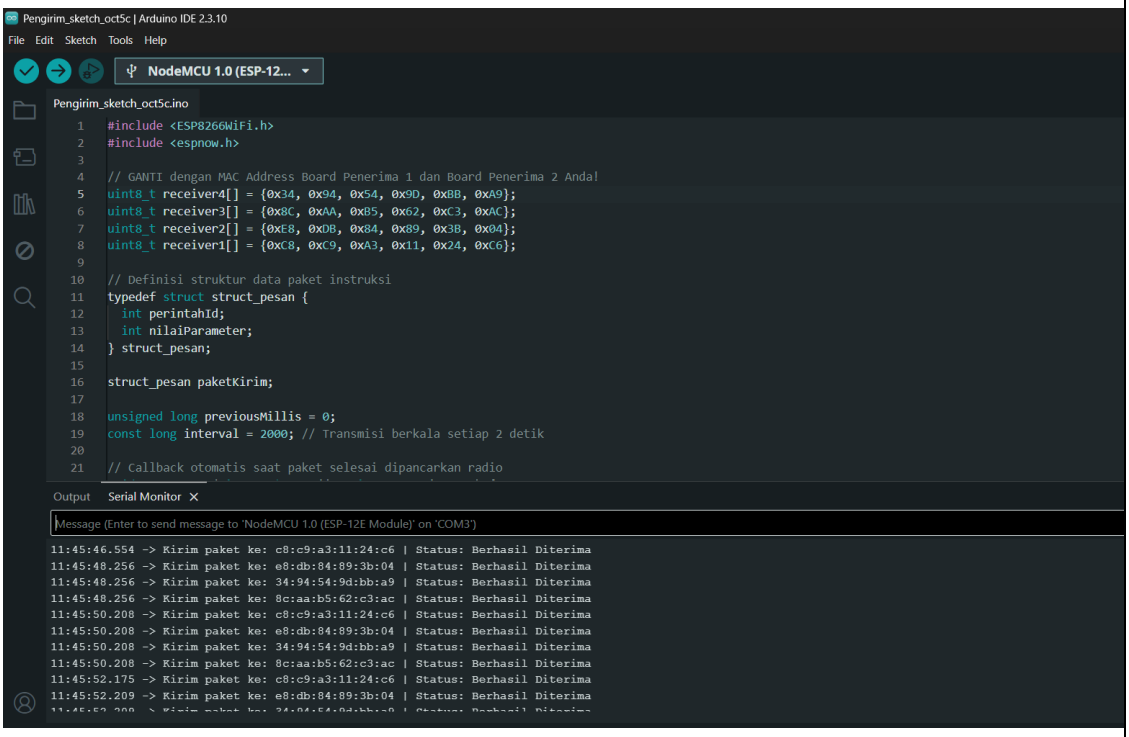


SS an Serial Monitor yang menampilkan MAC Address

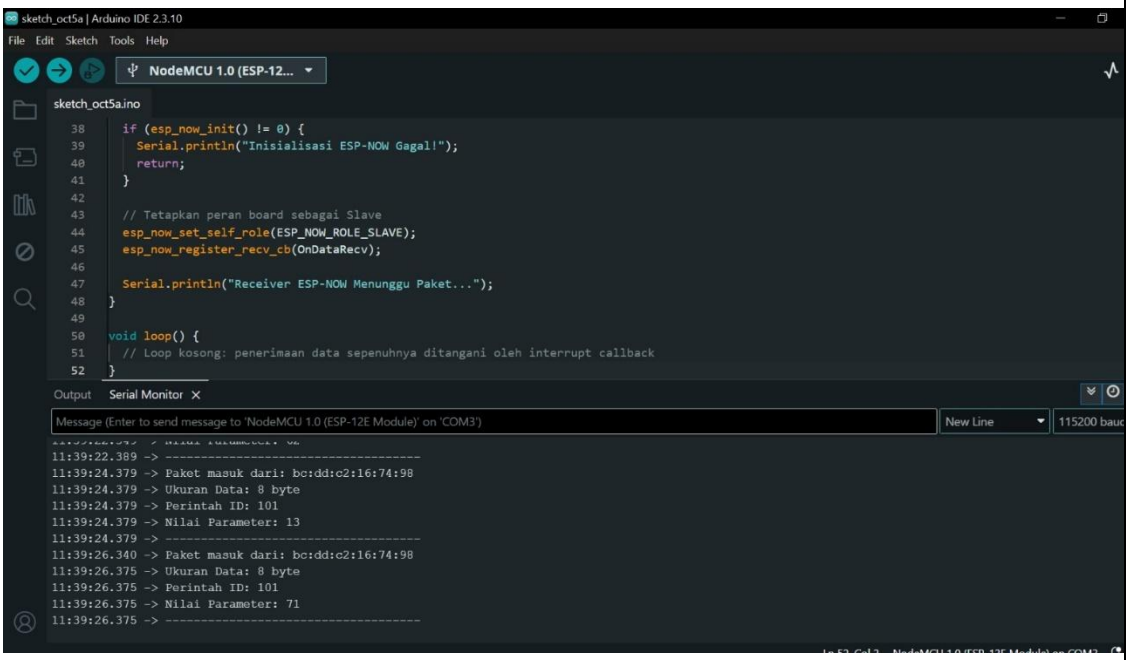


Salinan Di NotePadd MAC Addres kita

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

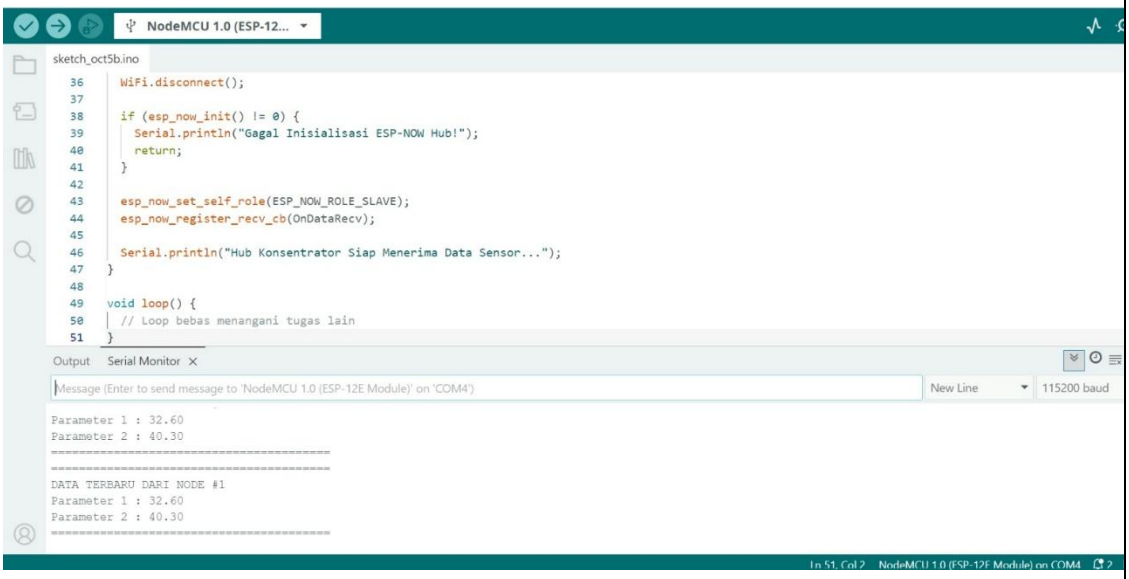


SS an Serial Monitor Pengirim (Server)

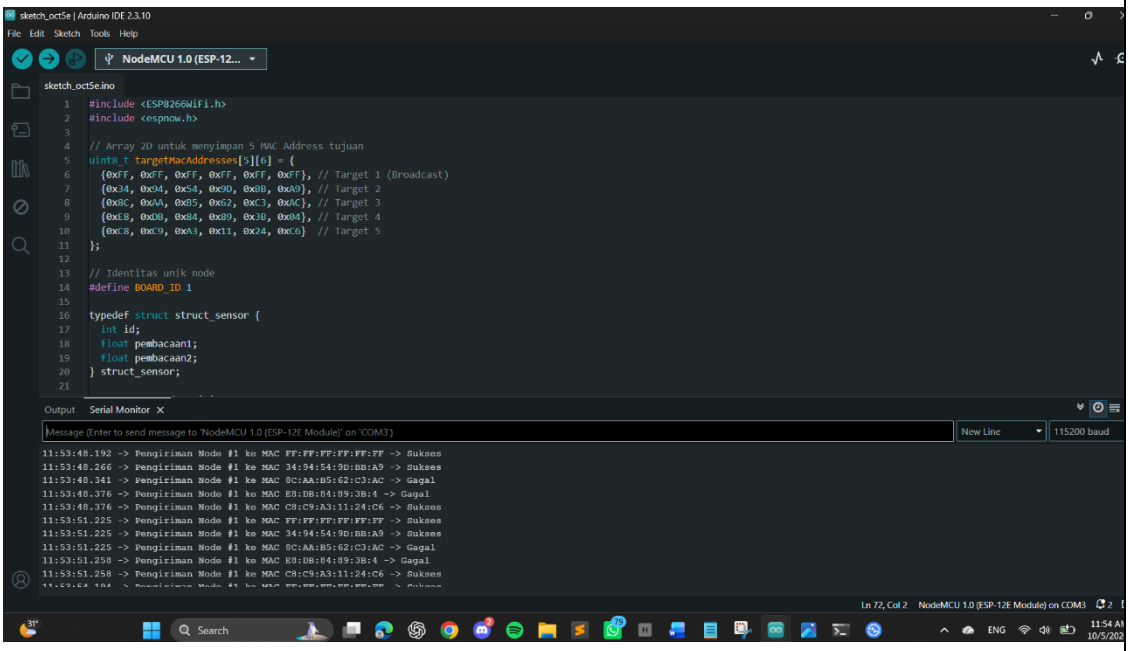


Serial Monitor Penerima

5.5.3 Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

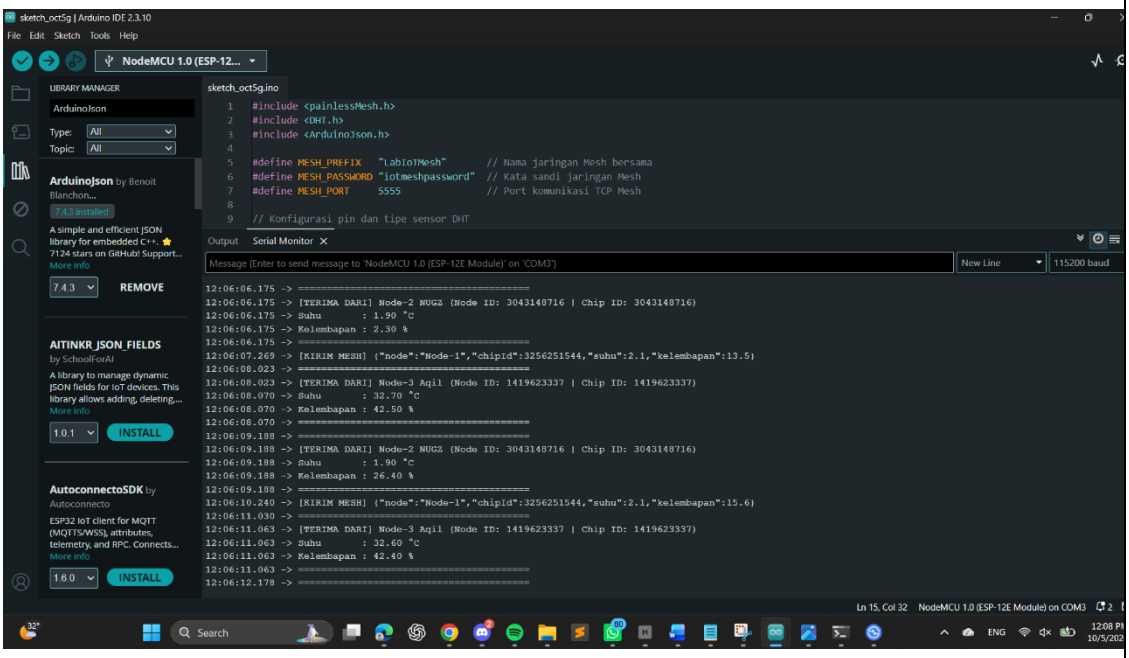


Serial Monitor Penerima



Serial Monitor Pengirim

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)



Tampilan Output Serial Monirot

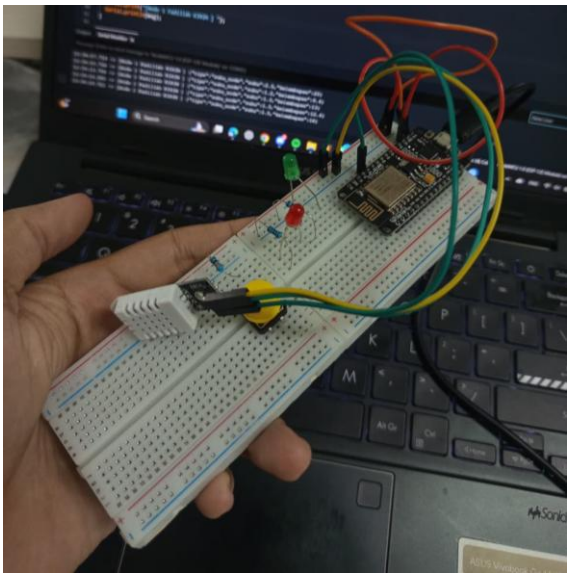


Foto Alat

5.6 Latihan

Kerjakan latihan berbasis eksperimen, observasi serial monitor, dan modifikasi kode berikut untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (sendStatus == 0), lakukan eksperimen pemutusan simpul:
    i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.

Jawab: **status pengiriman berubah.** Saat kabel USB Penerima (NodeMCU 2) dicabut, Serial Monitor pada NodeMCU Controller Pengirim yang tadinya mencetak Berhasil Diterima (*sendStatus == 0*) akan berganti mencetak **Gagal (Tidak Terjangkau)** (*sendStatus != 0*) spesifik untuk MAC Address milik NodeMCU 2.

    ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?

Jawab: **Ya, benar.** Status pengiriman di Serial Monitor NodeMCU Controller (pengirim) akan otomatis berganti dari yang tadinya Berhasil Diterima (atau status 0) menjadi **Gagal (Tidak Terjangkau)** khusus untuk MAC Address milik board yang kabel dayanya dicabut.

    iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Jawab: SP-NOW dapat mendeteksi kegagalan pengiriman tanpa koneksi TCP karena memanfaatkan **mekanisme *Hardware ACK (Acknowledgement)* bawaan standar Wi-Fi IEEE 802.11 pada Layer 2 (Data Link Layer / MAC Layer)**:

1. **Pengiriman Paket Sinyal (*Action Frame*):** Saat NodeMCU pengirim mengirim data ke MAC Address tujuan, data tersebut dibungkus dalam *Wi-Fi Action Frame* (Layer 2) langsung dari chip radio Wi-Fi.
2. **Tuntutan Balasan Hardware (*MAC Layer ACK*):** Secara spesifikasi Wi-Fi 802.11, ketika chip radio penerima berhasil menangkap paket data tersebut, **chip radio penerima wajib membalas secara otomatis** dengan sinyal *ACK Frame* dalam hitungan mikrodetik (proses ini dilakukan langsung oleh perangkat keras/hardware radio, tanpa menunggu kode program/TCP).
3. **Mekanisme Deteksi (*Timeout & Retry*):** Jika board penerima mati/dicabut, sinyal *ACK Frame* balasan tersebut tidak akan pernah terkirim. Chip radio pengirim akan mencoba melakukan pengiriman ulang (*retry*) beberapa kali.
4. **Triger Callback ke Software:** Ketika batas *retry* habis dan sinyal ACK tetap tidak diterima, chip radio ESP8266 melaporkan kegagalan pengiriman ini ke *firmware*, yang kemudian langsung memicu *callback function* OnDataSent di kode C++ untuk mencetak status **Gagal (Tidak Terjangkau)**.

- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID: ....
    i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* changedConnectionCallback() untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?

Jawab: Waktu yang dibutuhkan oleh *callback* changedConnectionCallback() untuk mendeteksi bahwa salah satu *node* telah keluar/terputus adalah sekitar 3 hingga 10 detik (rata-rata berkisar di 5 detik).

    ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Jawab: Waktu yang dibutuhkan hingga *node* tersebut kembali terdaftar secara otonom ke dalam topologi *mesh* adalah sekitar 3 hingga 8 detik setelah *board* selesai melakukan proses *booting* (total sekitar 5–12 detik dari saat daya disambungkan kembali).

- **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.
    i. Modifikasilah fungsi receivedCallback pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).

Jawab:

```
// Callback otomatis saat menerima pesan dari node manapun di jaringan mesh
void receivedCallback(uint32_t from, String &msg) {
  // Parsing muatan teks JSON yang diterima
  StaticJsonDocument<200> doc;
  DeserializationError error = deserializeJson(doc, msg);

  if (!error) {
    // Mengekstrak nilai dari JSON
    const char* sender = doc["node"];
    uint32_t chipId    = doc["chipId"];
    float suhu         = doc["suhu"];
    float kelembapan   = doc["kelembapan"];

    // RULE ENGINE: Filter Ambang Batas Suhu Kritis (> 32.0 C)
    if (suhu > 32.0) {
      // Cetak kartu telemetri penuh (Kondisi Kritis)
      Serial.println("=======================================");
      Serial.println("        [PERINGATAN SUHU KRITIS!]      ");
      Serial.printf("[TERIMA DARI] %s (Node ID: %u | Chip ID: %u)\n", sender, from, chipId);
      Serial.printf("Suhu       : %.2f °C\n", suhu);
      Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
      Serial.println("=======================================");
    } else {
      // Cetak status singkat satu baris (Kondisi Normal)
      Serial.printf("[NORMAL] Telemetri dari Node %u aman.\n", chipId);
    }

  } else {
    // Jika gagal parsing JSON
    Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from, msg.c_str());
  }
}
```

ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: [NORMAL] Telemetri dari Node <chipId> aman.

iii. Tuliskan blok fungsi receivedCallback(uint32_t from, String &msg) hasil modifikasi Anda!

- Jawab:

```
[NORMAL] Telemetri dari Node 2847192 aman.
[NORMAL] Telemetri dari Node 2847192 aman.
```

```
========================================
        [PERINGATAN SUHU KRITIS!]
[TERIMA DARI] Node-1 (Node ID: 2847192 | Chip ID: 2847192)
Suhu       : 33.40 °C
Kelembapan : 71.00 %
========================================
```

5.7 Tugas: Sistem Telemetri & Otomasi Terdistribusi Berbasis painlessMesh

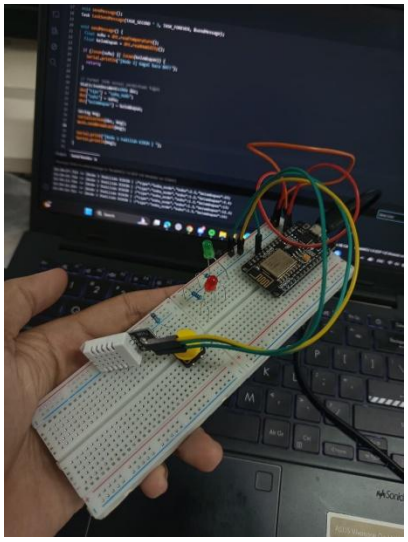
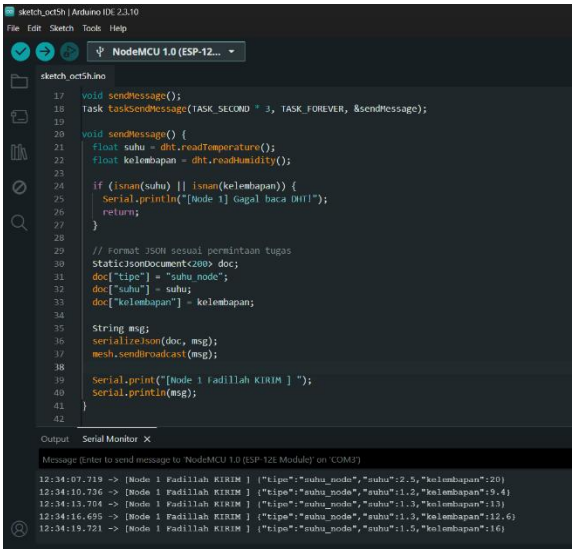


Foto Alat DHT



Serial Monitor DHT

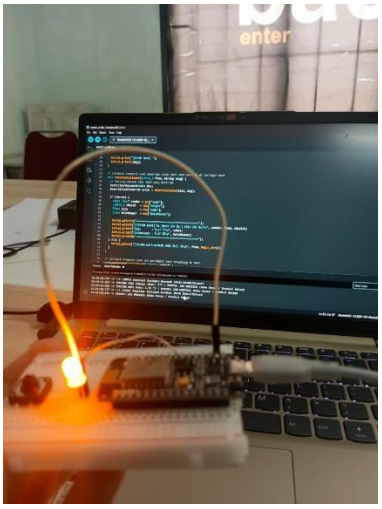
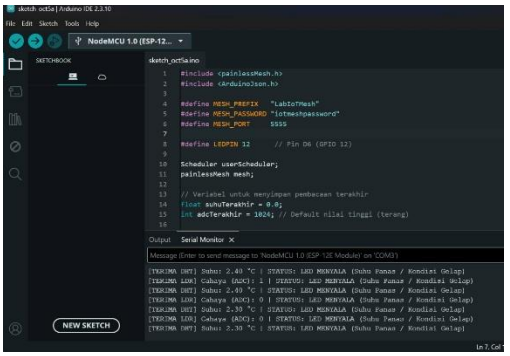
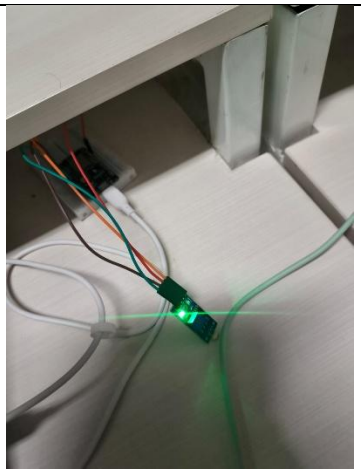
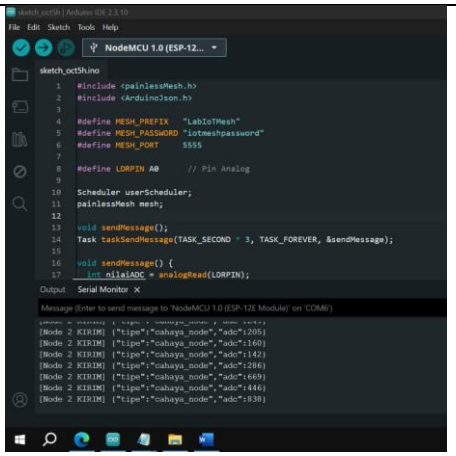


Foto Alat LED



Serial Monitor LED

	
Foto Alat LDR	Serial Monitor LDR